



**FACOLTÀ DI SCIENZE TECNOLOGICHE E  
DELL'INNOVAZIONE**

---

**CORSO DI LAUREA IN INGEGNERIA INFORMATICA (L-8)**

***Prova Finale in*  
PROGRAMMAZIONE**

Mercurio — Progettazione e implementazione di un'app iOS per  
l'aggregazione personalizzata di feed RSS italiani con integrazione di  
provider AI

**RELATORE**

Chiar.mo  
Prof. Mauro Bellone

**CANDIDATO**

Alessio Iodice  
MATR. 0082301723

Anno Accademico 2025/2026





# Sommario

Mercurio (nome in codice del progetto: *Mercury*, mantenuto per repository e identificatori interni) è un'applicazione iOS nativa per l'aggregazione di feed RSS italiani arricchiti con servizi di intelligenza artificiale. Il progetto è stato condotto con metodo ingegneristico: requisiti tracciabili, specifiche architetturali, modelli UML e diagrammi di sequenza. L'architettura adotta MVVM con SwiftUI, persistenza locale-first su SwiftData e un layer AI plug-in che astrae cinque provider esterni (OpenAI, Anthropic, Google, Ollama, DeepSeek). Il contributo metodologico distintivo sta nel flusso di sviluppo agentico: l'autore ricopre il ruolo di architetto e revisore, mentre l'implementazione è delegata a coppie umano-agente (Claude Code e Codex CLI) sotto governance documentale. La tesi descrive il sistema, ne discute il modello dei dati, le interfacce esterne e la pipeline di enrichment, e formalizza il processo di sviluppo agentico adottato. Lo strato dati è dimensionato per la demo con un catalogo di testate italiane; il supporto multi-regione, sviluppato nelle fasi iniziali, resta nel codice dietro un flag di build come possibile estensione futura.

**Codice sorgente.** Il repository del progetto è disponibile su GitHub: <https://github.com/Yoddikko/Mercury>.



# Indice

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>Sommario</b>                                    | <b>5</b>  |
| <b>1 Introduzione</b>                              | <b>11</b> |
| 1.1 Contesto e motivazione . . . . .               | 11        |
| 1.2 Obiettivi . . . . .                            | 11        |
| 1.3 Struttura del documento . . . . .              | 12        |
| 1.4 Glossario minimo . . . . .                     | 12        |
| <b>2 Stato dell'arte</b>                           | <b>15</b> |
| 2.1 Aggregatori di feed esistenti . . . . .        | 15        |
| 2.2 Tecniche di clustering di notizie . . . . .    | 15        |
| 2.3 Personalizzazione on-device . . . . .          | 16        |
| 2.4 LLM applicati a contenuti editoriali . . . . . | 16        |
| 2.5 Confronto sintetico . . . . .                  | 17        |
| <b>3 Analisi dei requisiti</b>                     | <b>19</b> |
| 3.1 Requisiti funzionali . . . . .                 | 19        |
| 3.2 Requisiti non funzionali . . . . .             | 19        |
| 3.3 Casi d'uso . . . . .                           | 19        |
| 3.4 Vincoli e ipotesi . . . . .                    | 21        |
| <b>4 Specifiche</b>                                | <b>25</b> |
| 4.1 Architettura logica . . . . .                  | 25        |
| 4.2 Modello del dominio . . . . .                  | 26        |
| 4.3 Interfacce esterne . . . . .                   | 27        |

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| 4.4      | Strategia di persistenza . . . . .                     | 28        |
| <b>5</b> | <b>Progettazione</b>                                   | <b>29</b> |
| 5.1      | Diagramma delle classi . . . . .                       | 29        |
| 5.2      | Diagrammi di sequenza . . . . .                        | 29        |
| 5.2.1    | Ingestion RSS . . . . .                                | 29        |
| 5.2.2    | Enrichment articolo . . . . .                          | 30        |
| 5.2.3    | Ranking feed personalizzato . . . . .                  | 30        |
| 5.2.4    | Apertura dettaglio articolo . . . . .                  | 31        |
| 5.3      | Schema dati SwiftData . . . . .                        | 31        |
| 5.4      | Strategia di estensibilità AI . . . . .                | 32        |
| <b>6</b> | <b>Metodologia di sviluppo agentic</b>                 | <b>35</b> |
| 6.1      | Approccio e principi . . . . .                         | 35        |
| 6.2      | Toolchain . . . . .                                    | 36        |
| 6.3      | Governance via documenti . . . . .                     | 36        |
| 6.4      | Flusso operativo . . . . .                             | 37        |
| 6.5      | Pattern ricorrenti e anti-pattern . . . . .            | 38        |
| 6.6      | Flusso di change-request guidato dall'utente . . . . . | 38        |
| 6.7      | Research-as-evidence . . . . .                         | 39        |
| 6.8      | Decomposizione multi-issue di una spec . . . . .       | 40        |
| 6.9      | Riflessione critica . . . . .                          | 42        |
| <b>7</b> | <b>Implementazione</b>                                 | <b>45</b> |
| 7.1      | Ingestione e catalogo RSS . . . . .                    | 45        |
| 7.2      | Persistenza locale (SwiftData) . . . . .               | 46        |
| 7.3      | Layer AI multi-provider . . . . .                      | 47        |
| 7.4      | Presentazione, orchestrazione e lettura . . . . .      | 47        |
| 7.5      | Lettura pulita: distillazione del corpo . . . . .      | 48        |
| 7.6      | Feed per argomenti e feed personalizzato . . . . .     | 49        |
| 7.6.1    | Onboarding e feed "Per te" . . . . .                   | 50        |
| 7.7      | Impostazioni, design system e osservabilità . . . . .  | 50        |

|          |                                   |           |
|----------|-----------------------------------|-----------|
| <b>8</b> | <b>Testing e validazione</b>      | <b>53</b> |
| 8.1      | Strategia di test . . . . .       | 53        |
| 8.2      | Copertura . . . . .               | 53        |
| 8.3      | Validazione qualitativa . . . . . | 54        |
| <b>9</b> | <b>Conclusioni</b>                | <b>55</b> |
| 9.1      | Risultati raggiunti . . . . .     | 55        |
| 9.2      | Limiti . . . . .                  | 56        |
| 9.3      | Sviluppi futuri . . . . .         | 56        |
|          | <b>Bibliografia</b>               | <b>57</b> |



# Capitolo 1

## Introduzione

### 1.1 Contesto e motivazione

Il consumo di notizie online vive due tensioni opposte. Da un lato la quantità di contenuti pubblicati ogni giorno supera di molto la capacità di attenzione di una persona. Dall'altro l'utente si ritrova frammentato su decine di applicazioni proprietarie, ognuna con il proprio algoritmo opaco e con poco controllo sulle fonti. RSS, pur tecnicamente datato, resta l'unico standard aperto che lascia all'utente la scelta esplicita delle fonti e la lettura in un unico luogo.

Nel frattempo i modelli linguistici di grandi dimensioni (LLM) hanno reso economicamente sostenibile l'arricchimento automatico dei contenuti editoriali: sintesi, categorizzazione, clustering. Gli aggregatori commerciali attuali hanno però due limiti. L'arricchimento è spesso opaco e legato a un singolo fornitore, e i dati di interazione finiscono in cloud anche quando non ce ne sarebbe bisogno.

Mercury nasce per esplorare un punto intermedio: un aggregatore RSS nativo iOS in cui (i) la persistenza è locale per impostazione predefinita, (ii) l'utente sceglie quale provider AI usare e con quali credenziali, e (iii) sintesi e categorizzazione sono trasparenti, etichettate e disattivabili.

### 1.2 Obiettivi

La tesi persegue cinque obiettivi tra loro complementari:

1. progettare un'architettura modulare iOS che separi presentazione, dominio, persistenza e integrazioni AI in layer testabili;
2. realizzare una pipeline RSS robusta capace di ingestire, normalizzare e deduplicare articoli da fonti eterogenee;
3. integrare cinque provider AI (OpenAI, Anthropic, Google, Ollama, Deep-Seek) dietro un'unica astrazione, con configurazione utente;
4. garantire una persistenza locale-first con SwiftData per supportare l'uso offline e la privacy;
5. formalizzare e validare un **processo di sviluppo agentico** documentato, con tracciabilità completa issue → branch → PR → merge, come contributo metodologico oltre al manufatto software.

## 1.3 Struttura del documento

Il documento segue la catena ingegneristica classica Sommerville, 2016 richiesta dal relatore. Il Cap. 2 delinea lo stato dell'arte. Il Cap. 3 formalizza requisiti funzionali e non funzionali con identificatori tracciabili. Il Cap. 4 traduce i requisiti in specifiche architetturali e di dominio. Il Cap. 5 contiene il modello UML, lo schema dati e i diagrammi di sequenza. Il Cap. 6 è il contributo distintivo della tesi: describe toolchain, governance documentale e riflessione critica sull'approccio agentico. Il Cap. 7 riporta le scelte implementative non banali con riferimento alle issue e ai PR del repository. Il Cap. 8 discute strategia e copertura dei test, mentre il Cap. 9 sintetizza risultati, limiti e sviluppi futuri.

## 1.4 Glossario minimo

Termini ricorrenti nel documento, definiti una volta sola.

**Outlet / testata** Una fonte giornalistica del catalogo (es. ANSA, Corriere). Un outlet può esporre più feed RSS (sezioni).

**Feed RSS** Il file XML pubblicato da una testata con l'elenco degli ultimi articoli (titolo, link, riassunto, data).

**Ingest** Il momento in cui l'app scarica i feed e salva gli articoli in locale.

**Enrichment** Il passo successivo all'ingest: l'app scarica la pagina web dell'articolo per ricavarne il testo completo, perché il feed di solito contiene solo un estratto.

**Chrome / boilerplate** Tutto ciò che nella pagina web non è articolo: banner cookie, menu, box “Leggi anche”, pulsanti di condivisione, inviti all'abbonamento.

**Distillazione** La pulizia automatica della pagina: dal HTML completo si estrae il solo corpo dell'articolo, eliminando il chrome. È una pipeline a stadi successivi.

**Pipeline** Una catena di passi applicati in sequenza allo stesso dato; l'output di un passo è l'input del successivo.

**CTA** *Call to action*: testo o pulsante che invita l'utente a fare qualcosa (“Abbonati”, “Accetta i cookie”).

**Teaser** L'anteprima gratuita di un articolo a pagamento: poche righe seguite dall'invito ad abbonarsi.

**Fixture** Una pagina HTML reale salvata nel repository e usata nei test: la pipeline viene rieseguita su di essa a ogni build per accorgersi subito di eventuali regressioni.

**Readability** Algoritmo open source di Mozilla (usato dalla modalità lettura di Firefox e da molte app di lettura) che individua il corpo di un articolo dentro una pagina web.

**Purge / retention** Pulizia della cache locale: *purge* = cancellare subito ciò che non serve più; *retention* = conservare i dati solo per un periodo massimo (qui 30 giorni).



# Capitolo 2

## Stato dell'arte

### 2.1 Aggregatori di feed esistenti

Il segmento degli aggregatori RSS è dominato da pochi prodotti consolidati. *Feedly* offre sincronizzazione cloud, categorizzazione automatica e da qualche anno una funzione di sintesi basata su LLM (Feedly AI) disponibile nelle fasce a pagamento Feedly, 2024. *NetNewsWire* è un client open source per Apple Simmons e NetNewsWire contributors, 2024, strettamente locale, privo di funzioni AI integrate. *Reeder* è un client commerciale focalizzato sulla qualità di lettura e sulla compatibilità con servizi di sincronizzazione di terze parti (iCloud, Feedbin, Inoreader). Ne emerge una separazione netta tra “letture pulite ma senza intelligenza” (NetNewsWire, Reeder) e “intelligenza opaca lato cloud” (Feedly). Mercury si colloca in mezzo: letture pulite locali, arricchimento AI esplicito e configurabile.

### 2.2 Tecniche di clustering di notizie

La letteratura sul clustering di notizie include sia approcci industriali proprietari sia sistemi accademici. *Google News* costruisce *storylines* aggregando articoli su uno stesso evento attraverso similarità testuale e segnali temporali. Sistemi accademici come *NewsClusterer* e variazioni di Louvain Blondel et al., 2008 o

HDBSCAN Campello et al., 2013 su embedding densi mostrano che il clustering online (incrementale, su singolo articolo in arrivo) è praticabile su volumi medi.

Tre sono le scelte che contano: la rappresentazione (TF-IDF contro embedding neurale), la metrica di similarità (la cosine è lo standard de facto Manning et al., 2008) e la soglia di assegnazione. Mercury adotta la specifica per un clustering opzionale basato su embedding e cosine similarity con soglia 0,75, come documentato in `docs/features/CLUSTERING.md`. L'implementazione è rimandata a fasi successive del progetto.

## 2.3 Personalizzazione on-device

La personalizzazione di contenuti su dispositivo attira interesse crescente perché evita il trasferimento di segnali comportamentali sensibili. Apple ha introdotto framework dedicati (*Core ML*, *Create ML*, *Natural Language*) Apple Inc., 2024a che consentono inferenza locale su modelli moderatamente grandi. Per task “leggeri” come il ranking di un feed personale (centinaia di articoli, decine di feature) un punteggio lineare locale basta e avanza, e ha il vantaggio di restare ispezionabile.

Il principio guida è *local-first* Kleppmann et al., 2019: i dati di interazione restano sul dispositivo, la sincronizzazione cloud è opt-in. Mercury vi aderisce: SwiftData è la sorgente di verità, Firebase è citato come opzione futura per la sincronizzazione cross-device.

## 2.4 LLM applicati a contenuti editoriali

L'uso di LLM per sintesi, categorizzazione e classificazione di testi giornalistici è ormai consolidato. I rischi documentati sono soprattutto le allucinazioni (introduzione di fatti non presenti nel testo originale) Ji et al., 2023, il bias del modello e la dipendenza da un singolo fornitore commerciale. Le mitigazioni accettate includono prompt deterministici con vincoli espliciti (“usa solo il contenuto fornito”),

output JSON validati lato client, e astrazione del fornitore dietro un'interfaccia comune così da poter migrare o duplicare senza riscrivere la logica applicativa.

Sono esattamente le mitigazioni che Mercury adotta: prompt versionati (`docs/ai/PROMPTS.md`), astrazione `AIProvider` (PR #23) e quattro implementazioni intercambiabili.

## 2.5 Confronto sintetico

La Tab. 2.1 riassume il posizionamento di Mercury rispetto agli aggregatori principali, lungo le dimensioni rilevanti per gli obiettivi del progetto.

Tabella 2.1: Confronto sintetico tra Mercury e gli aggregatori RSS principali.

| Dimensione                          | Feedly           | NetNewsWire | Reeder | Mercury              |
|-------------------------------------|------------------|-------------|--------|----------------------|
| Open source                         | no               | sì          | no     | sì (MIT)             |
| Storage primario                    | cloud            | locale      | locale | locale               |
| Sintesi AI integrata                | sì (a pagamento) | no          | no     | sì (on-demand)       |
| Categorizzazione AI                 | sì (opaca)       | no          | no     | sì (etichettata)     |
| Provider AI scegliibile dall'utente | no               | —           | —      | sì (5 provider)      |
| Clustering eventi                   | parziale         | no          | no     | specificato (futuro) |
| Funziona offline                    | limitato         | sì          | sì     | sì                   |
| Etichettatura contenuto AI          | parziale         | —           | —      | richiesta            |



# Capitolo 3

## Analisi dei requisiti

### 3.1 Requisiti funzionali

I requisiti funzionali sono numerati e prioritizzati secondo lo schema MoSCoW DSDM Consortium, 2014 (M = Must, S = Should, C = Could). La priorità riflette lo scope per la versione MVP della tesi.

### 3.2 Requisiti non funzionali

### 3.3 Casi d'uso

La Fig. 3.1 riassume i casi d'uso del sistema. Le tabelle che seguono descrivono i quattro scenari principali.

#### **UC1: configurazione preferenze (onboarding)**

**Attore:** lettore (primo avvio).

**Pre:** app installata, nessun profilo locale.

**Flusso principale:** quattro passi. Introduzione alle funzioni, selezione delle testate italiane (senza scaricare articoli), scelta degli interessi (etichette suggerite più campo libero), configurazione facoltativa del provider AI.

**Post:** esiste un `UserPreferenceEntity` locale con `hasCompletedOnboarding`

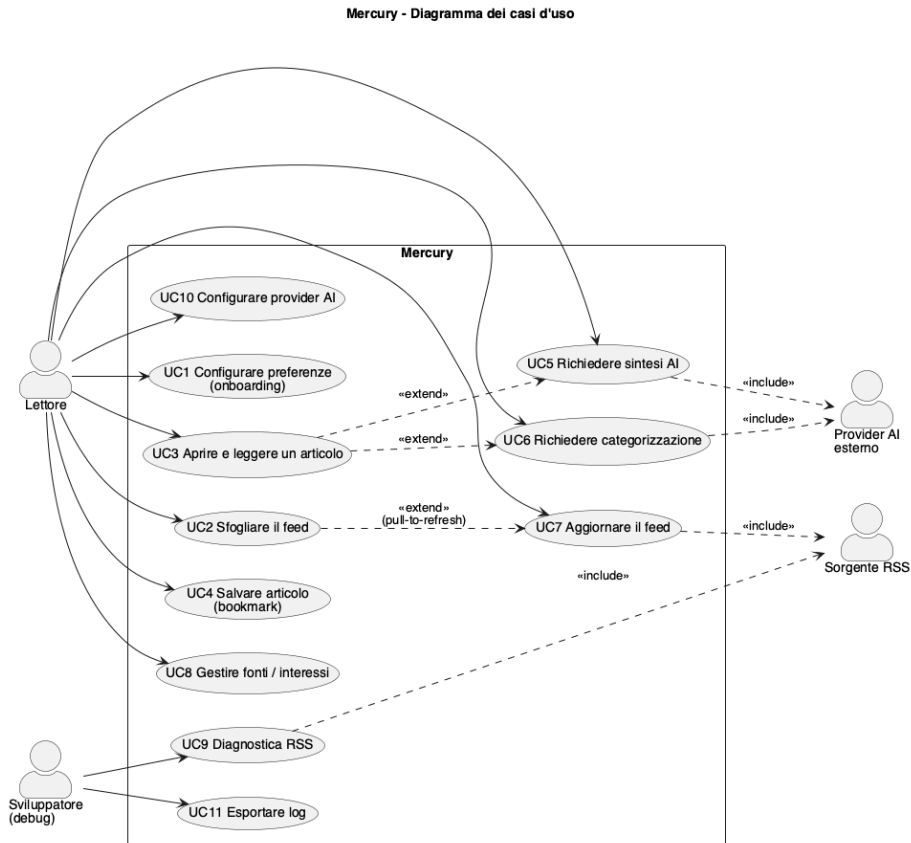


Figura 3.1: Diagramma dei casi d'uso di Mercury.

attivo; il primo download parte solo dopo la conferma finale.

**Tracciabilità:** RF-15, RF-19, RF-21.

## UC2: lettura del feed

**Attore:** lettore.

**Flusso principale:** apertura Home, caricamento articoli da SwiftData, visualizzazione in lista, pull-to-refresh facoltativo (estende UC7). La Home offre tre viste: cronologica, per argomenti (sez. 7.6) e personalizzata “Per te” (sez. 7.6.1).

**Variante:** senza interessi o senza provider AI configurato la vista “Per te” mostra uno stato dedicato; il fallback resta il feed cronologico (RF-5).

**Tracciabilità:** RF-4, RF-5, RF-16, RF-21.

### UC3: sintesi AI di un articolo

**Attore:** lettore + provider AI esterno.

**Pre:** articolo aperto, AI configurata, connettività disponibile.

**Flusso principale:** richiesta sintesi, costruzione prompt, chiamata provider, validazione output JSON, persistenza locale, render con etichetta “generato da AI”.

**Eccezioni:** provider non configurato → stato vuoto con messaggio; errore provider → log e UI non bloccante.

**Tracciabilità:** RF-10, RF-11, RF-12, RF-14.

### UC4: salvataggio articolo

**Attore:** lettore.

**Flusso principale:** tap su pulsante bookmark, aggiornamento campo `isBookmarked` su `ArticleEntity`, visibilità persistente offline.

**Tracciabilità:** RF-8.

## 3.4 Vincoli e ipotesi

- Target tecnologico: iOS 17+, Swift 5.9+, SwiftData, SwiftUI.
- Approccio local-first: la fonte di verità è on-device; il cloud è opzionale e non richiesto dall’MVP.
- L’utente fornisce le proprie credenziali API. Mercury non distribuisce token né opera un backend proxy nella versione MVP.
- I provider AI sono assunti remoti (eccetto Ollama, locale). La latenza è gestita tramite operazioni asincrone e non blocca la lettura.
- Il clustering semantico su embedding è specificato come obiettivo; l’MVP ne implementa un’approssimazione lessicale on-device (sez. 7.6).

**Tracciabilità.** Ogni RF è ricondotto al capitolo che lo realizza nella colonna *Cap.* di Tab. 3.1. Le specifiche del Cap. 4 dettagliano i componenti che soddisfano

i requisiti; il Cap. 7 cita le issue e i PR specifici (es. RF-1/2/3 → PR #16, #22, #38; RF-4 → PR #24, #35, #37; RF-5 → PR #36; RF-6/8 → PR #35, #40; RF-9 → PR #15, #20, #41; RF-10/11 → PR #23, #39, #42; RF-15 → PR #24, #34; RF-18 → PR #19).

Tabella 3.1: Requisiti funzionali (RF).

| ID    | Descrizione                                                                                                                                   | Pri. | Cap. |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------|------|------|
| RF-1  | Il sistema deve scaricare feed RSS/Atom da un catalogo di sorgenti predefinite.                                                               | M    | 7    |
| RF-2  | Il sistema deve effettuare il parsing dei feed scaricati estraendo metadati estesi (titolo, link, autore, data, immagine, lingua, categorie). | M    | 7    |
| RF-3  | Il sistema deve normalizzare gli articoli in un modello di dominio unico e deduplicare per URL.                                               | M    | 7    |
| RF-4  | Il sistema deve persistere gli articoli normalizzati in modo locale e offline.                                                                | M    | 7    |
| RF-5  | Il sistema deve esporre un feed di articoli in ordine cronologico inverso come comportamento di fallback.                                     | M    | 7    |
| RF-6  | L'utente deve poter aprire il dettaglio di un articolo e accedere al corpo completo.                                                          | M    | 7    |
| RF-7  | Se il corpo RSS appare incompleto, il sistema deve effettuare un best-effort di estrazione dalla pagina HTML originale.                       | S    | 7    |
| RF-8  | L'utente deve poter salvare (bookmark) un articolo.                                                                                           | S    | 7    |
| RF-9  | Il sistema deve esporre un Developer Playground in modalità debug per diagnostica dei feed.                                                   | S    | 7    |
| RF-10 | Il sistema deve fornire un layer AI plug-in compatibile con almeno cinque provider (OpenAI, Anthropic, Google, Ollama, DeepSeek).             | M    | 7    |
| RF-11 | L'utente deve poter selezionare provider, modello e fornire la propria API key tramite Keychain.                                              | M    | 7    |
| RF-12 | Il sistema deve generare una sintesi (corta + bullet) di un articolo su richiesta utente.                                                     | S    | 7    |
| RF-13 | Il sistema deve categorizzare un articolo (categoria singola + tag multipli) su richiesta.                                                    | S    | 7    |
| RF-14 | Ogni contenuto generato da AI deve essere etichettato come tale nell'interfaccia.                                                             | M    | 7    |
| RF-15 | Il sistema deve memorizzare preferenze utente (categorie, topic, fonti preferite, fonti nascoste, lingua).                                    | S    | 4    |
| RF-16 | Il sistema deve produrre un feed personalizzato ordinato per punteggio (recency, match categoria, match topic, engagement, preferenza fonte). | C    | 7    |
| RF-17 | Il sistema deve poter raggruppare articoli sullo stesso evento (cluster) tramite similarità su embedding.                                     | C    | 7    |

Tabella 3.2: Requisiti non funzionali (RNF).

| ID     | Categoria      | Vincolo                                                                                                                             |
|--------|----------------|-------------------------------------------------------------------------------------------------------------------------------------|
| RNF-1  | Prestazioni    | Il rendering della home non deve bloccare il main thread; le operazioni di rete e AI devono essere asincrone (async/await).         |
| RNF-2  | Prestazioni    | La lettura del feed da cache locale deve essere percepita come istantanea (<100 ms per centinaia di articoli).                      |
| RNF-3  | Privacy        | Nessun dato di interazione utente lascia il dispositivo per impostazione predefinita. La sincronizzazione cloud è opt-in.           |
| RNF-4  | Sicurezza      | Le API key dei provider AI devono essere memorizzate in Keychain, mai in chiaro su disco.                                           |
| RNF-5  | Sicurezza      | I log non devono contenere segreti, token o contenuto utente privato.                                                               |
| RNF-6  | Offline        | Articoli, sintesi, categorie e preferenze già acquisite devono essere disponibili offline.                                          |
| RNF-7  | Accessibilità  | Tutti i controlli interattivi devono avere etichette di accessibilità localizzate.                                                  |
| RNF-8  | Localizzazione | Le stringhe rivolte all'utente non devono essere hard-coded; uso obbligatorio dei file di localizzazione.                           |
| RNF-9  | Manutenibilità | Architettura MVVM rispettata; nessuna chiamata a provider AI direttamente da Views/ViewModels.                                      |
| RNF-10 | Osservabilità  | Ogni funzione nuova o modificata deve emettere log via <code>AppLogger</code> ; propagazione del request id nei flussi multi-layer. |
| RNF-11 | Robustezza     | Nessun fallimento silenzioso: errori RSS, AI o persistenza devono produrre log e fallback.                                          |
| RNF-12 | Portabilità    | Target iOS 17+, Swift 5.9+, dipendenze esterne minime (Swift-Data, SwiftUI nativi).                                                 |

# Capitolo 4

## Specifiche

### 4.1 Architettura logica

Mercury adotta MVVM con SwiftUI Apple Inc., 2024c per la presentazione, una application layer di use case che orchestra i servizi di dominio, una data layer che isola persistenza e networking, e una AI layer plug-in che astrae i provider esterni. La Fig. 4.1 riporta i componenti principali e le loro dipendenze.

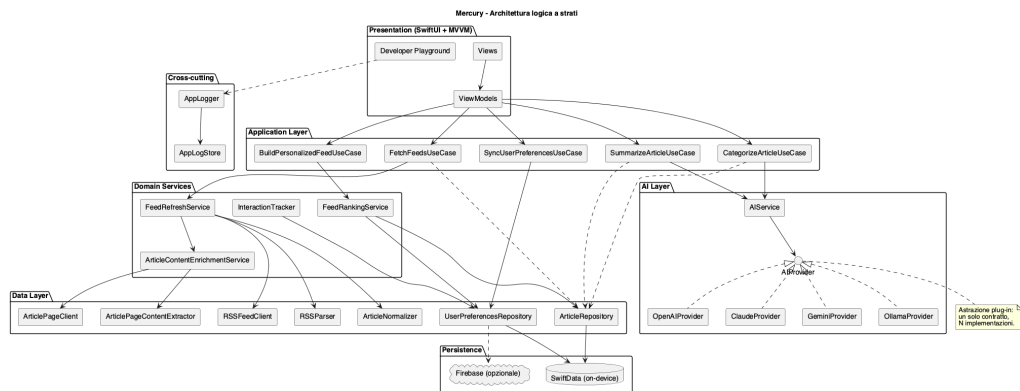


Figura 4.1: Architettura logica di Mercury (componenti e dipendenze).

#### Principi di separazione.

- le View non contengono logica di parsing, rete o persistenza;
- i ViewModel non chiamano direttamente i provider AI;

- ogni use case incapsula uno scenario applicativo e orchestra repository e servizi;
- l'unico punto di ingresso verso un provider AI è `AIService`, che non effettua fallback automatici (fail-fast).

## 4.2 Modello del dominio

Il modello di dominio è composto da quattro entità persistenti.

**ArticleEntity.** È un articolo normalizzato. Campi chiave: `id` (hash dell'URL), `title`, `sourceName`, `sourceURL`, `articleURL`, `publishedAt`, `rawContent`, `cleanedContent`, `summaryShort`, `summaryBullets`, `category`, `tags`, `language`, `isBookmarked`, `isRead`, `clusterID`, `createdAt`, `updatedAt`. È la *single source of truth* per il contenuto. Tutti i campi AI sono opzionali e popolati in modo asincrono e incrementale.

**ClusterEntity.** È un evento, cioè un gruppo di articoli che parlano della stessa notizia. Campi: `id`, `title`, `summary`, `mainTopic`, `articleIDs`, `createdAt`. Un articolo può appartenere ad al più un cluster.

**UserPreferenceEntity.** Profilo locale dell'utente. Campi: `preferredCategories`, `preferredTopics` (gli interessi del feed “Per te”), `favoriteSources`, `hiddenSources`, `preferredLanguage`, `enabledRegionRawValues`, `hasCompletedOnboarding`, `updatedAt`. È prodotto al termine dell'onboarding e modificato dalle Impostazioni.

**InteractionEntity.** Evento di interazione utente. Campi: `articleID`, `actionType` (`opened`, `bookmarked`, `ignored`, `shared`), `timestamp`, `readingDuration`, `scrollDepth`. Alimenta gli inferred interests per il ranking personalizzato.

## 4.3 Interfacce esterne

**Sorgenti RSS.** Mercury consuma feed conformi a RSS 2.0 RSS Advisory Board, 2009 e Atom (RFC 4287) Nottingham e Sayre, 2005. Il parser estrae, quando disponibili: link canonico, GUID/id, data e lingua, autore (`author`, `dc:creator`), categorie, immagine principale (`media:*`, `enclosure`, `itunes:image`, fallback su primo `<img>`), corpo completo (`content:encoded`, Atom `content`) e summary di fallback.

**Provider AI.** Tutti i provider implementano l'interfaccia `AIProvider`. I provider supportati sono:

- *OpenAI*: endpoint REST, autenticazione tramite Bearer token;
- *Anthropic Claude*: endpoint REST, autenticazione tramite header `x-api-key`;
- *Google Gemini*: endpoint REST, autenticazione tramite API key in query string;
- *Ollama*: endpoint locale configurabile, token opzionale;
- *DeepSeek*: endpoint REST OpenAI-compatibile, autenticazione tramite Bearer token (PR #39).

Ogni provider richiede la configurazione esplicita di un modello (nessun default implicito) e un timeout. Le credenziali sono in Keychain.

**Estrazione contenuto pagina.** Per i feed che pubblicano solo abstract, il sistema scarica la pagina HTML originale e applica un estrattore euristico per recuperare il corpo principale e l'immagine eroe.

**Sintesi AI on-demand.** Coerentemente con RF-12, la sintesi dell'articolo è esposta in dettaglio come azione esplicita avviata da interazione utente (pulsante dedicato), non come step automatico della pipeline di ingestion né dell'apertura dettaglio. La scelta è formalizzata dalla PR #48 (`docs(summarization)`):

change AI summary trigger to on-demand button) che ha aggiornato docs/features/SUMMARIZATION.md come fonte di verità; il refactor del trigger lato codice è tracciato dalla issue #47. Le ragioni sono due: (i) controllo esplicito del costo per chiamata al provider, (ii) contenuto AI etichettato e prodotto solo su intento dell'utente (RF-14).

## 4.4 Strategia di persistenza

SwiftData Apple Inc., 2024b è la sorgente di verità on-device. Tutte le entità sono modelli @Model interrogati tramite ModelContext. Le scelte progettuali sono:

- **Upsert per URL hash:** l'id dell'articolo è derivato dall'URL così da rendere idempotente l'ingestion;
- **Soft-delete non previsto:** gli articoli sono soggetti a politiche di retention basate su data, non a tombstones;
- **AI cache nei campi articolo:** summaryShort, summaryBullets, category, tags vivono sulla stessa riga dell'articolo, evitando join e migrazioni separate.

La sincronizzazione cloud (Firebase) è prevista come estensione opzionale per le entità di profilo (UserPreferenceEntity, riferimenti InteractionEntity aggregati) ma non per il corpus articoli, che resta locale.

# Capitolo 5

## Progettazione

### 5.1 Diagramma delle classi

La Fig. 5.1 riporta il modello UML delle entità persistenti e dei principali servizi di dominio e AI. Le molteplicità rilevanti sono indicate sulle associazioni. Lo schema riflette l'estensione di `ArticleEntity` introdotta dalla PR #37 (campi `externalID`, `authorName`, `heroImageURL`, `contentSource`, `contentWordCount`, `isContentLikelyComplete`), il quinto provider `AI DeepSeekProvider` (PR #39) e i nuovi servizi di persistenza `ArticleLocalStore` (PR #35) e `UserPreferencesService` (PR #34) introdotti per il wiring della Home (PR #36).

### 5.2 Diagrammi di sequenza

#### 5.2.1 Ingestion RSS

La sequenza di Fig. 5.2 formalizza il flusso di refresh. Il `HomeViewModel` attiva l'use case, che delega a `FeedRefreshService`. Per ciascun feed si esegue fetch HTTP, parsing RSS/Atom, normalizzazione e deduplicazione intra-feed. A valle del loop si applica una deduplicazione cross-feed e si esegue un upsert atomico sul repository. Ogni fallimento di fetch o parsing viene loggato e non interrompe la pipeline; il report finale espone successi e fallimenti per outlet.

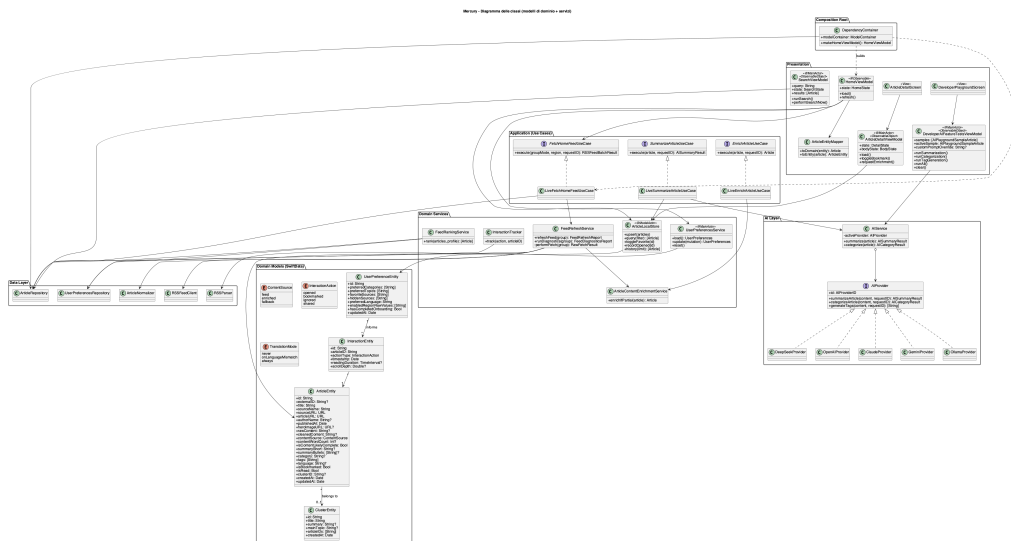


Figura 5.1: Diagramma delle classi: modelli di dominio e servizi, aggiornato allo schema esteso post PR #37 con isolamento via `@ModelActor` per lo store articoli e `@MainActor` per il servizio preferenze.

## 5.2.2 Enrichment articolo

L’arricchimento (Fig. 5.3) è on-demand e incrementale. Sintesi e categorizzazione possono procedere in parallelo perché operano sullo stesso `cleanedContent` ma scrivono campi disgiunti. `AIService` è fail-fast: in caso di errore non tenta un secondo provider, ma propaga l’eccezione al chiamante, che decide se ritentare. Così si evitano comportamenti sorprendenti dovuti a switch silenziosi.

## 5.2.3 Ranking feed personalizzato

La Fig. 5.4 descrive il flusso di costruzione del feed personalizzato come *specifica*. La specifica include il calcolo del punteggio per articolo, controlli di diversità (evitare concentrazione su singola fonte o micro-topic) e un fallback cronologico in caso di insuccesso del ranking. L’implementazione la realizza in due forme: il ranking a punteggio del feed cronologico (sez. 7.4) e il feed “Per te” basato sugli interessi dichiarati e classificato dal provider AI (sez. 7.6.1); la personalizzazione sui segnali di lettura impliciti resta specifica.

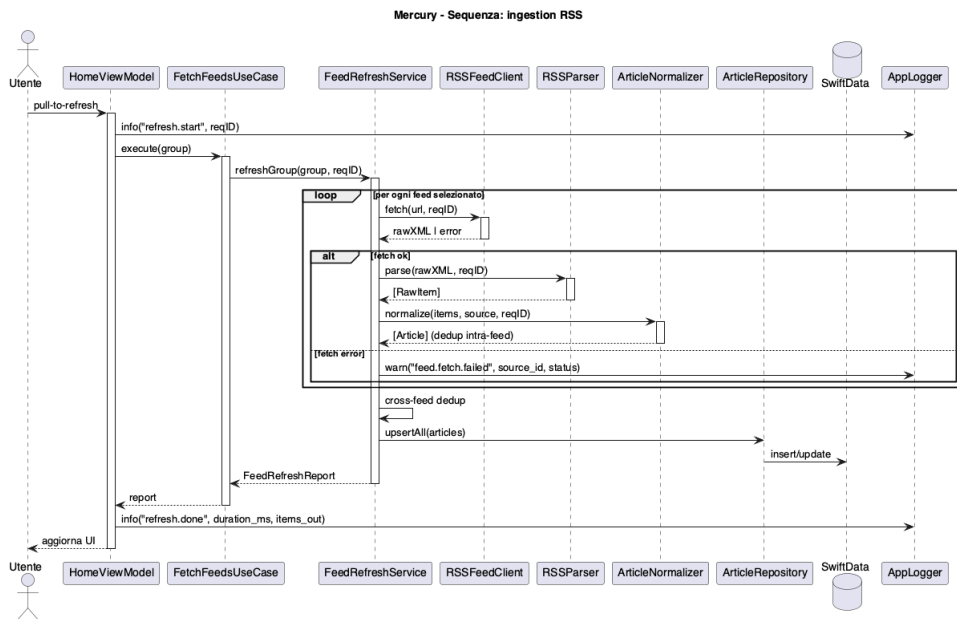


Figura 5.2: Sequenza di ingestion RSS.

## 5.2.4 Apertura dettaglio articolo

La Fig. 5.5 formalizza l'apertura del dettaglio articolo dalla Home, realizzata in PR #40 (issue #26). Il tap sulla riga produce una `NavigationLink` verso `ArticleDetailScreen`; il `ArticleDetailViewModel` (`@MainActor, ObservableObject`) gestisce una macchina a stati `idle`  $\rightarrow$  `loading`  $\rightarrow$  `loaded`  $\rightarrow$  `error` e invoca `ArticleLocalStorage` per `recordOpen` e `toggleFavorite`. Lo slot di enrichment del corpo è una closure iniettabile `EnrichContent?`: il servizio di sintesi AI (issue #28 e successive) vi è stato poi cablato senza modificare il contratto del `ViewModel`, come previsto.

## 5.3 Schema dati SwiftData

La Tab. 5.1 sintetizza le entità persistenti e le loro relazioni. Le chiavi primarie sono tutte `String` (UUID per profilo/interazioni, hash URL per articoli/cluster).

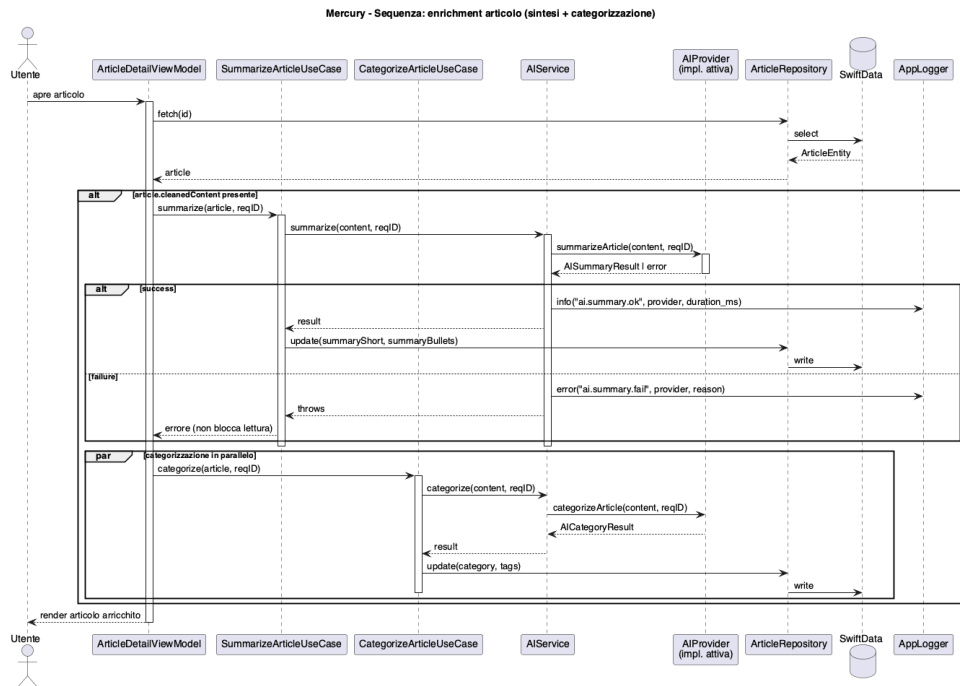


Figura 5.3: Sequenza di enrichment di un articolo.

Tabella 5.1: Entità SwiftData e relazioni principali.

| Entità               | Chiave / Identità            | Relazioni                        |
|----------------------|------------------------------|----------------------------------|
| ArticleEntity        | id = hash(articleURL)        | clusterID → ClusterEntity (0..1) |
| ClusterEntity        | id = UUID                    | articleIDs: lista riferimenti    |
| UserPreferenceEntity | id = UUID (singleton locale) | —                                |
| InteractionEntity    | id = UUID                    | articleID → ArticleEntity        |

## 5.4 Strategia di estensibilità AI

L'astrazione `AIProvider` (Listato 5.1) è il punto di estensione del layer AI. Un nuovo provider richiede: (i) implementare il protocollo, (ii) registrare l'implementazione nel `DependencyContainer`, (iii) esporre la configurazione (modello, timeout, endpoint) nelle preferenze sviluppatore, (iv) aggiornare la documentazione in `docs/ai/PROVIDERS.md`.

Listing 5.1: Astrazione `AIProvider` (estratto, fonte `docs/architecture/ARCHITECTURE.md`).

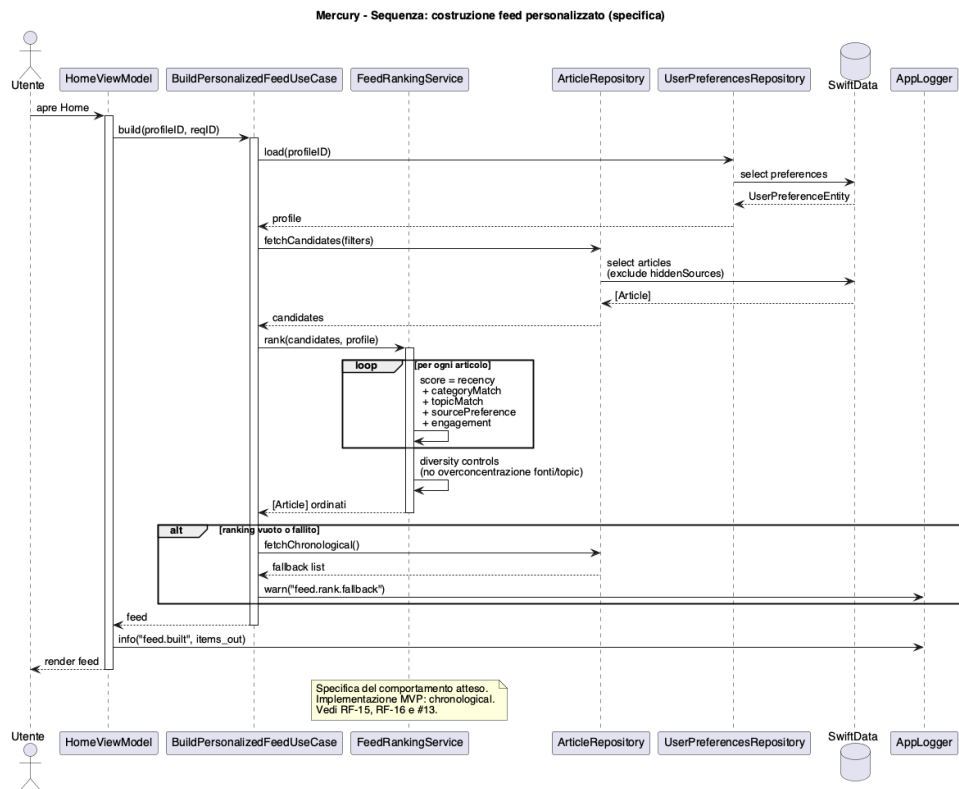


Figura 5.4: Sequenza di costruzione del feed personalizzato (specifica).

```

protocol AIPProvider {
    var id: AIPProviderID { get }

    func summarizeArticle(_ content: String,
        requestId: String?) async throws
        -> AISummaryResult

    func categorizeArticle(_ content: String,
        requestId: String?) async
        throws -> AICategoryResult

    func generateTags(_ content: String,
        requestId: String?) async throws ->
        [String]
}
  
```

La selezione del provider attivo avviene tramite AIService, che legge le preferenze (provider id, modello, token in Keychain) e instrada le chiamate.

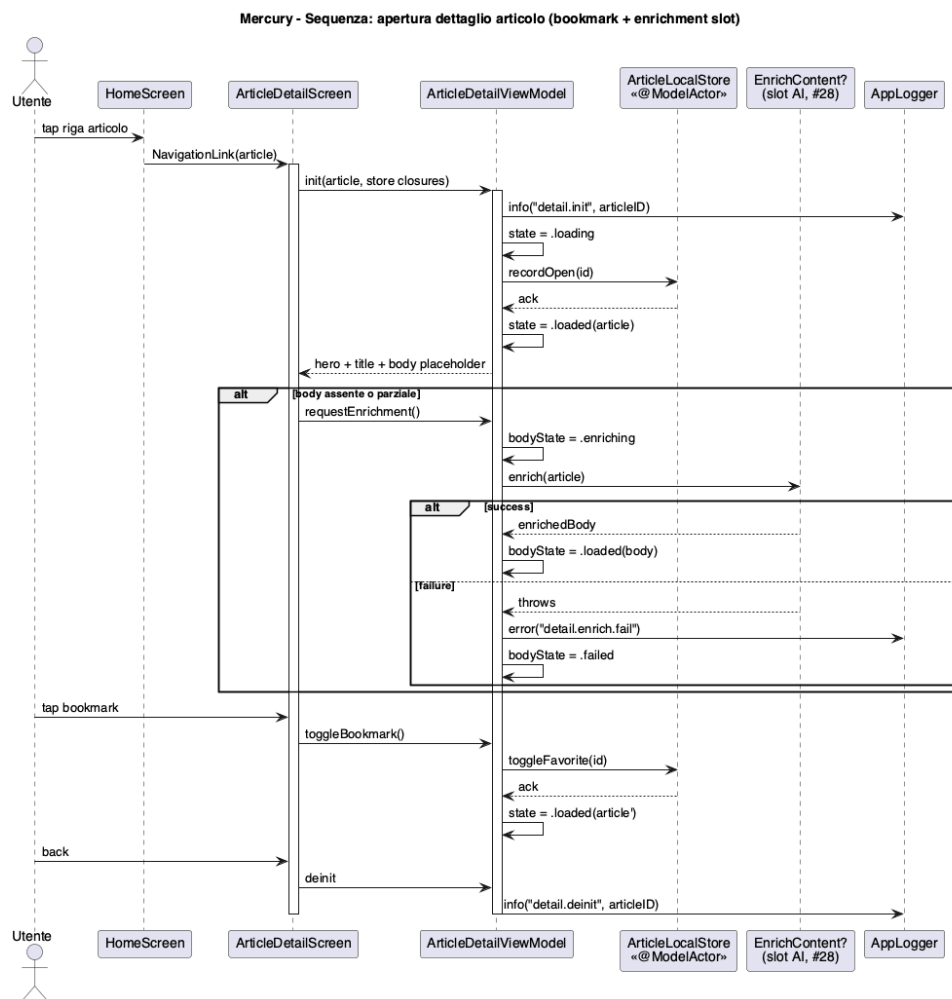


Figura 5.5: Sequenza di apertura dettaglio articolo (PR #40): navigazione, recordOpen, toggle bookmark e slot di enrichment.

AIService non implementa fallback automatico tra provider: in caso di errore il chiamante riceve l'eccezione e decide se ritentare o cambiare provider esplicitamente. La scelta è documentata in `docs/ai/PROVIDERS.md` come *fail-fast by design*.

# Capitolo 6

## Metodologia di sviluppo agentico

### 6.1 Approccio e principi

Mercury è stato sviluppato secondo un modello che chiamiamo *human-as-architect*, *AI-as-implementer*. All’umano spetta definire requisiti, specifiche, scope di ogni issue, criteri di accettazione e revisione finale. All’agente spetta tradurre l’intento in codice, nel rispetto di una governance documentale vincolante. Il presupposto è che l’agente, se opportunamente vincolato, sia un implementatore affidabile su task a scope stretto, mentre l’umano resta responsabile della coerenza architetturale, della valutazione dei trade-off e dell’integrazione nel disegno complessivo.

I principi adottati sono tre:

1. **Scope ristretto per issue.** Ogni issue definisce un cambiamento coerente e verificabile (una feature, una correzione, una specifica). Le issue “grandi” vengono spezzate prima della delega.
2. **Governance via documenti.** Le regole stanno nei file `AGENTS.md`, `docs/guidelines/CLAUDE.md`, `docs/guidelines/CODEX.md` e `docs/guidelines/*`. Gli agenti li leggono come istruzioni vincolanti prima di operare.
3. **CI come rete di sicurezza.** Build iOS, test unitari, controllo di sincronia tra docs e codice, lint markdown e check di localizzazione girano su ogni PR. Una PR rossa non viene mergiata.

## 6.2 Toolchain

La toolchain di sviluppo è composta da:

- **Claude Code** (Anthropic): agente CLI usato per task che richiedono ragionamento architetturale, refactoring multi-file e revisione di codice esistente;
- **Codex CLI** (OpenAI): agente CLI usato per task implementativi più diretti a scope stretto, tipicamente associati a issue numerate;
- **GitHub Actions**: automazione CI per build, test, lint e check di consistenza docs;
- **gh CLI**: interfaccia GitHub usata sia dagli agenti per aprire PR e leggere review, sia dall'umano per orchestrare il flusso.

Il criterio di scelta empirico è semplice: Codex per l'implementazione mirata su una issue chiusa, Claude per l'analisi e il refactoring trasversale. Le branch prodotte da Codex seguono lo schema `codex/issue-N-<slug>`, visibile nello storico git (es. `codex/issue-7-swiftdata-models`, `codex/issue-21-rss-enrich`).

## 6.3 Governance via documenti

Il file `AGENTS.md` alla radice del repository elenca le letture obbligatorie e le regole inderogabili. Estratto:

*“documentation is the source of truth . . . prefer the smallest coherent change . . . if behavior, contracts, models, or architecture change, update docs in the same work session . . . if new or modified functions are introduced, add or update logs using AppLogger.”*

Il file `docs/guidelines/CODEX.md` dettaglia gli ambiti in cui un agente può modificare il codice senza intervento umano:

*“Codex is allowed to introduce small implementation-driven changes when necessary . . . the change must be necessary . . . minimal . . . consistent with the architecture . . . the related docs must be updated.”*

La governance comprende sotto-policy specifiche e vincolanti: `TESTING.md` (livelli di test attesi per categoria di cambiamento), `LOCALIZATION.md` (nessuna stringa rivolta all'utente hard-coded), `LOGGING.md` (log obbligatori in ogni funzione nuova o modificata, propagazione del `request_id`), `GIT_WORKFLOW.md` (Conventional Commits Conventional Commits, 2023, scope coerente, no commit a meno di richiesta esplicita).

L'effetto pratico è che il *system prompt* di ogni agente non è una stringa magica dentro la sessione: è un insieme di file nel repository, versionato, revisionabile e modificabile come qualunque altro artefatto di progettazione.

## 6.4 Flusso operativo

Il flusso canonico è formalizzato nella Fig. 6.1 ed è composto dai seguenti step:

1. l'umano apre una issue su GitHub che funge da specifica (cosa, perché, criteri di accettazione, riferimenti ai docs);
2. l'umano invoca un agente passando il numero della issue e indicandogli di leggere `AGENTS.md` e `docs/guidelines/*`;
3. l'agente crea una branch `codex/issue-N-<slug>`, esegue le modifiche al codice, ai test, ai log, alle stringhe localizzate e ai docs;
4. l'agente apre una PR compilando il template (impatto su docs, test, localizzazione);
5. la CI esegue build, test, lint e check di sincronia;
6. l'umano effettua la review architetturale; se ok, merge;
7. eventuali rifiniture post-merge ai docs avvengono nello stesso ciclo di lavoro.

## 6.5 Pattern ricorrenti e anti-pattern

### Pattern che hanno funzionato.

- *Issue come specifica chiusa*: più la issue è puntuale, più il risultato dell'agente è prevedibile (es. #17 outlets export, #18 logging, #7 SwiftData models).
- *Guidelines come prompt implicito*: mettere `LOGGING.md` e `LOCALIZATION.md` tra le letture obbligatorie ha eliminato classi intere di follow-up correttivi.
- *Doc-as-code*: la regola “cambia comportamento → aggiorna docs nello stesso commit” ha evitato il drift specifica/codice.
- *CI scope-limited*: workflow leggeri (lint markdown, controllo sincronia) trovano rapidamente le incoerenze prima della review umana.

### Anti-pattern osservati.

- *Scope ambiguo*: issue formulate come “migliorare RSS” hanno prodotto modifiche fragili, che poi hanno richiesto interventi correttivi (cfr. PR #22 seguito dai fix `f7acbb4` e `8678155`).
- *Refactor opportunistici*: tentativi di rifattorizzazioni non richieste dentro PR di feature; mitigati dalla regola “no unrelated refactors”.
- *Optimismo nei test*: in alcune PR i test sono stati dichiarati non necessari senza giustificazione; ora il template di PR li richiede esplicitamente.

## 6.6 Flusso di change-request guidato dall'utente

Il flusso canonico issue → branch → PR descritto nella sezione precedente copre il caso in cui la specifica è stabile e l'implementazione segue. Il caso opposto, una *change-request* dell'utente che modifica una specifica già implementata, è gestito da una variante distinta, formalizzata nelle linee guida di progetto. Il flusso ha cinque step rigidi:

1. **Spec change:** l'umano aggiorna i docs rilevanti (`docs/features/*.md`, `docs/ai/*.md`, `docs/architecture/ARCHITECTURE.md`) come fonte di verità, con modifica minima e chirurgica alla sola sezione interessata;
2. **Issue creation:** si apre una issue che linka la sezione docs modificata e descrive scope, anti-scope e PR eventualmente da revertire o correggere;
3. **Implementation:** l'agente opera in worktree isolato con brief stretto, anti-scope esplicito e commit atomici;
4. **Merge:** dopo CI verde e review, squash + delete branch.

La regola inderogabile è “docs prima, codice dopo”: mai modificare il comportamento e poi i docs. Il primo esempio canonico è la modifica del trigger di sintesi AI da automatico a on-demand, mergiata in PR #48 (`docs(summarization): change AI summary trigger to on-demand button`) in ~5 s con scope esclusivamente documentale (8 file modificati, 551 inserzioni). L'issue #47 traccia il refactor di codice conseguente, che resta un delta limitato perché la specifica è già fissata nei docs.

Il contributo metodologico ha due facce. La prima: separando lo “spec change” (rapido, documentale, revisionabile) dall’“implementation” (soggetta a CI, test, review) si rende esplicito il momento in cui un requisito è stato *deciso*, distinto dal momento in cui viene *realizzato*. La seconda: il flusso evita la classe di errore più costosa del modello agentico, cioè l'agente che reinterpreta uno scope ambiguo durante l'implementazione. Quando la spec è già scritta sui docs e linkata dalla issue, il margine interpretativo collassa e l'output dell'agente diventa prevedibile.

## 6.7 Research-as-evidence

Per i task di lavoro profondo in cui l'output non è codice ma una *base di conoscenza* (per esempio il censimento di outlet RSS, il mapping dei provider AI, l'indagine di formati eterogenei), il flusso canonico è stato raffinato in un pattern

dedicato: il deliverable di ricerca viene committato come *artefatto JSON versionato* **prima** che l’implementazione lo consumi. Committare l’artefatto rende auditabile la provenienza di ogni decisione successiva e disaccoppia la fase di harvesting dalla fase di porting nel codice.

Il caso canonico è la PR #52 (issue #50), che ha esteso il catalogo RSS a 228 outlet su 25 regioni. La fase di ricerca (Phase A) ha prodotto `docs/rss/research/candidates-202` 233 candidati con provenienza per outlet (URL, homepage, slant editoriale, lingua, fonte di harvesting tra otto OPML pubblici e registri di settore, eventuale flag `verify` per gli endpoint fragili). L’agente di Phase A è morto al cinquantesimo tool-use per un rate-limit del provider, ma il JSON era già materializzato su disco: il curatore umano lo ha committato come `b2cd647`, così che un agente fresco di Phase B potesse riprendere da una base stabile e portare il catalogo nello strato Swift senza rieseguire la ricerca. Il fallimento a metà lavoro è diventato osservabile e recuperabile in pochi minuti, invece di un evento da contenere.

Il pattern generalizza il principio già discusso nella riflessione sui fallimenti osservati (sez. precedente): i checkpoint materializzati su disco e versionati sopravvivono alla morte dell’agente. Rispetto al normale commit incrementale cambia la natura del checkpoint, che qui non è codice di transizione ma un deliverable di ricerca autosufficiente, citato in seguito sia dai commit di porting (`fa1545f`, `c2f7acc`) sia dai docs per-regione (`ITALIAN_RSS.md`, `EUROPE_RSS.md`, `AMERICAS_RSS.md`, `ASIA_RSS.md`) come fonte unica di verità. La validazione effettiva degli endpoint (Phase 3) è deferita all’issue #53 per non confondere lo stadio di censimento con quello di healthcheck.

## 6.8 Decomposizione multi-issue di una spec

Il flusso di change-request (sez. 6.6) gestisce il caso in cui una singola modifica documentale si traduce in un solo PR di implementazione. Serve invece una variante più articolata quando la change-request introduce una capacità nuova che attraversa più moduli e merita un percorso di review separato per ciascuno. Il caso canonico è la richiesta del 30 giugno 2026 “rendering del corpo articolo con

possibilità di scegliere tra web view e nativo” (issue #51), che ha esercitato per la prima volta il pattern.

Il flusso ha aggiunto due caratteristiche al modello a cinque step:

1. *Spec change documentale con scope architetturale* (PR #56): aggiornamento di tre file (`docs/features/ARTICLE.md` con la sezione “Body Rendering”, `docs/features/SETTINGS.md` come nuovo feature doc, `docs/architecture/ARCHITECTURE.md` con il modulo §6.5). La spec ha incluso anche la *decisione di libreria* (SwiftSoup contro Kanna/Fuzi) motivata dalla copertura di edge case dell’HTML reale e dalla licenza Apache-2.0 compatibile con il vincolo thesis-friendly.
2. *Decomposizione in N issue* con dipendenze esplicite (#57 WebView default, #58 Settings + toggle persistito, #59 renderer nativo opt-in via SwiftSoup). Ogni issue ha citato la sezione di docs di pertinenza come autoritativa.
3. *Stacked PR*: PR #60 contro `main`, PR #61 contro il branch di #60, PR #62 contro il branch di #61. Lo stack mantiene la review per-modulo (un PR è verificabile in isolamento) preservando la dipendenza tecnica (il compilatore vede l’intera catena sul branch finale).
4. *Merge sequenziale con rebase automatico*: il merge di #60 ha chiuso i branch dipendenti, costretto un rebase di #61 su `main` (le commit di #60 sono state riconosciute come già applicate dal cherry-pick detection di git e silenziosamente saltate), e ripetuto con #62. La chiusura del chain ha richiesto la riapertura tecnica come PR #63 e #64 dopo che GitHub ha auto-chiuso #61 e #62 alla cancellazione del loro branch base. Il chain di tre PR è stato chiuso senza conflitti reali, perché ogni rebase ha solo riconosciuto contenuti identici a quelli appena squashed.

Il contributo metodologico è una separazione esplicita tra *spec atomica* (una sola PR documentale, mergiabile in pochi secondi) e *implementazione segmentata* (più PR di codice, ciascuna reviewable da un umano in pochi minuti). La somma rispetta sia il vincolo di non accumulare PR monolitiche sia quello di

non frammentare la spec in più documenti che potrebbero divergere. Il prezzo è la complessità del merge: il rebase automatico funziona perché il contenuto del PR base è *identico* dopo lo squash, ma una review che richieda modifiche al PR intermedio costringe a rebase manuali a cascata. Il pattern regge per stack di due o tre PR; oltre quella soglia conviene una decomposizione laterale (issue indipendenti che convergono in un solo PR di wiring).

## 6.9 Riflessione critica

**Limiti.** (1) *Lock-in al tooling*: la dipendenza da agenti commerciali (Claude, Codex) introduce un rischio di continuità in caso di cambio di pricing o policy. Mitigazione adottata: il prompt è il repository stesso, quindi migrabile su qualunque agente che sappia leggere `AGENTS.md`. (2) *Allucinazioni*: gli agenti tendono a inventare API o pattern non esistenti nel codice quando lo scope è ampio. Mitigazione: scope ristretto e CI rigida. (3) *Debito invisibile*: gli agenti producono codice plausibile che può accumulare debito non subito percepibile (over-engineering, astrazioni inutili). Mitigazione: review umana focalizzata sull'over-engineering, non solo sulla correttezza.

**Rischi.** (1) *Drift specifica/implementazione* se il vincolo “doc-as-code” viene rilassato in fretta sotto pressione. (2) *Falsa sicurezza dalla CI*: una build verde non implica che il design sia corretto. (3) *Trasferibilità*: il modello human-as-architect richiede un umano con competenze architetture; non sostituisce la formazione.

**Mitigazioni adottate.** Tutte le regole di processo sono documentate in modo esplicito nel repository, versionate e oggetto di review come il codice. La tracciabilità verso le issue e i PR, mantenuta nel capitolo di metodologia e nelle tabelle dei requisiti, rende il percorso di progetto verificabile end-to-end.

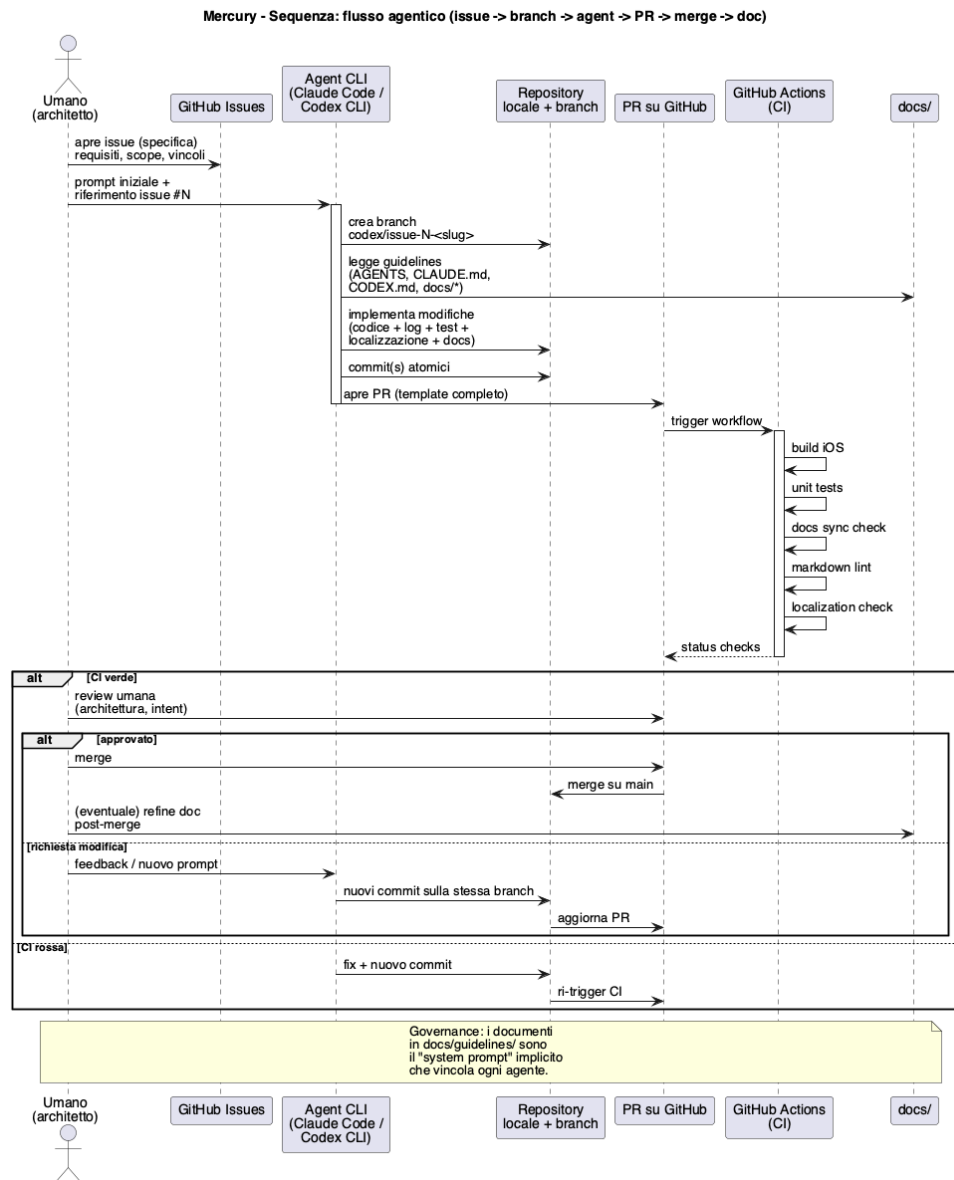


Figura 6.1: Flusso agentico canonico: issue → branch → agent → PR → CI → review → merge.



# Capitolo 7

## Implementazione

Questo capitolo descrive il sistema *come realizzato*, in corrispondenza delle specifiche del Cap. 4 e della progettazione del Cap. 5. L'accento è sui componenti e sulle decisioni ingegneristiche non banali; il percorso di sviluppo (issue, revisioni, iterazioni) è discusso nel Cap. 6.

### 7.1 Ingestione e catalogo RSS

L'ingestione è una pipeline a stadi: un client HTTP scarica i feed, un parser RSS/Atom ne estrae i metadati estesi (titolo, link canonico, GUID, data, lingua, autore, categorie, immagine principale, corpo), un normalizzatore li riconduce al modello di dominio e deduplica per URL. L'identificatore dell'articolo deriva dall'hash dell'URL, così l'upsert è idempotente senza lookup preliminari. Il `FeedRefreshService` espone due superfici sullo stesso core privato: un percorso di produzione, che restituisce un report minimo orientato al rendering, e un percorso di diagnostica, riservato al Developer Playground, con timing e payload grezzi. Ogni errore di fetch o parsing viene loggato e non interrompe il ciclo: il report finale espone successi e fallimenti per singola testata.

Il catalogo delle fonti nasce come censimento multi-regione (Tab. 7.1), ma lo scope del prodotto è ristretto alle sole testate italiane, dove si è concentrato il lavoro sulla qualità di lettura. Le sezioni non italiane restano nel sorgente ma sono escluse dalla compilazione tramite un flag di build (`MERCURY_MULTI_REGION`):

riattivarle è una modifica di una riga, non una riscrittura. Con una sola regione l'interfaccia non mostra alcuna selezione geografica e l'onboarding elenca direttamente le testate italiane. In questa fase l'app assume il nome utente *Mercurio* (l'identificatore interno resta *Mercury*).

Tabella 7.1: Distribuzione degli outlet del censimento multi-regione. In produzione è attiva la sola area italiana.

| Area                                              | Outlet     |
|---------------------------------------------------|------------|
| Italia                                            | 58         |
| Resto Europa (UK, FR, DE, ES, e altre 17 regioni) | 138        |
| Stati Uniti                                       | 15         |
| Brasile                                           | 9          |
| Giappone                                          | 8          |
| <b>Totale</b> su 25 regioni                       | <b>228</b> |

**Validazione del catalogo.** Un “feed sano” non è solo un feed che risponde: da ogni feed deve essere possibile scaricare un articolo recente e ricavarne il testo. Con questo criterio l'intero catalogo italiano è stato validato dal vivo, rimuovendo le voci non funzionanti (archivi congelati, proxy che restituiscono pagine senza testo, feed che servono solo la schermata di consenso). Le poche voci degradate ma ancora utili sono annotate; il report è mantenuto in `docs/rss/VALIDATION.md`.

## 7.2 Persistenza locale (SwiftData)

SwiftData Apple Inc., 2024b è la sorgente di verità on-device. Le quattro entità del Cap. 4 (`ArticleEntity`, `ClusterEntity`, `UserPreferenceEntity`, `InteractionEntity`) sono modelli `@Model` con mapping da/verso il dominio. L'accesso agli articoli passa da `ArticleLocalStore`, un `@ModelActor` che incapsula il `ModelContext` e centralizza le invarianti (deduplica per id, ordinamento temporale, retention), restituendo al chiamante solo tipi valore

già convertiti. Le preferenze sono gestite da `UserPreferencesService` (`@MainActor`) con semantica di singleton locale. Le migrazioni sono additive: i campi nuovi sono opzionali, così aggiungere un attributo non invalida i record già scritti.

## 7.3 Layer AI multi-provider

Il layer AI astrae cinque provider (OpenAI, Anthropic Claude, Google Gemini, Ollama, DeepSeek) dietro un unico protocollo `AIProvider; AIService` è l'unico punto di ingresso e instrada le chiamate in base alle preferenze (provider, modello, credenziali in Keychain). La scelta è *fail-fast*: in caso di errore l'eccezione risale al chiamante, senza fallback silenzioso verso un altro provider. La deserializzazione delle risposte è centralizzata in un unico punto, che mappa ogni anomalia di formato su un errore tipizzato e ripara i casi ricorrenti dei modelli (JSON troncato dal budget di token, virgole pendenti, output avvolto nel testo di ragionamento): una sola correzione vale per tutti e cinque i provider. Aggiungere un provider significa implementare il protocollo e registrarlo, riusando lo scaffolding OpenAI-compatibile dove applicabile.

## 7.4 Presentazione, orchestrazione e lettura

La Home è governata da `HomeViewModel`, una macchina a stati osservabile (`idle`, `loading`, `loaded`, `failed`). Il comportamento è *offline-first*: all'apertura si carica subito lo snapshot in cache e, solo se la rete è disponibile, un refresh in background riconcilia via upsert. Tra presentazione e servizi si interpone un layer di *use case* (`FetchHomeFeed`, `EnrichArticle`, `SummarizeArticle`) che isola la logica di dominio dai `ViewModel`, coerentemente con i principi di separazione delle specifiche; ogni use case espone un seam di test a closure, invece di introdurre un protocollo per ogni singola dipendenza.

Il dettaglio articolo (`ArticleDetailScreen + ArticleDetailViewModel`) rende corpo e metadati con una macchina a stati e slot iniettabili per gli effetti

collaterali (bookmark, registrazione dell’apertura, arricchimento). Il corpo è reso da un **lettore nativo** SwiftUI: l’HTML sanitizzato è parsato in blocchi tipizzati (paragrafo, titolo, immagine, lista, citazione, codice) con SwiftSoup Hedley, 2024 e reso con primitive native; l’HTML malformato ricade su un paragrafo di fallback senza generare crash. Una precedente modalità WebView, con relativo interruttore nelle Impostazioni, è stata rimossa a favore del solo lettore nativo: con la distillazione (sez. 7.5) che consegna già un HTML pulito, un secondo percorso di rendering era complessità senza beneficio.

**Ranking del feed.** Un servizio di dominio puro riordina gli articoli con uno scoring additivo e ispezionabile a partire dalle preferenze (categoria, topic, sorgente favorita, lingua), filtra le sorgenti nascoste e usa la freschezza come tie-break. In assenza di segnali dell’utente diventa un pass-through esplicito: il feed resta cronologico. Non c’è un protocollo dietro il servizio finché esiste un solo algoritmo, per non introdurre astrazioni speculative.

**Ricerca locale.** La ricerca testuale opera sugli articoli in cache con debouncing e cancellazione cooperativa. Il filtro testuale è applicato in-memory dopo il fetch persistente, perché comporlo dentro il `#Predicate` di `SwiftData` non è affidabile; il costo è trascurabile sul corpus dell’MVP e la scelta è isolata in un solo metodo, reversibile quando la piattaforma lo supporterà.

## 7.5 Lettura pulita: distillazione del corpo

Molte testate consegnano, via RSS o via pagina HTML, un corpo carico di rumore non editoriale (banner cookie, sidebar “Leggi anche”, strisce di condivisione, inviti alla newsletter, immagine hero duplicata). La distillazione è una pipeline a stadi, ispirata a Mozilla Readability Mozilla, 2024 e costruita su SwiftSoup senza nuove dipendenze: rimozione del boilerplate per token di classe/id noti e per densità di link; deduplicazione dell’immagine hero nel corpo; rimozione di frasi ricorrenti localizzate (per l’italiano: “riproduzione riservata”, “leggi anche”, disclaimer cookie). L’output è persistito con un numero di versione del distillatore, così gli

articoli in cache vengono rielaborati quando la pipeline evolve; una catena di fallback garantisce che il risultato non sia mai peggiore del testo di partenza.

**Regole per-testata e paywall.** La distillazione generica non coglie il rumore specifico di ogni testata. Un file di regole per-dominio (dati, non codice) indica per ciascuna testata dove sta il corpo, cosa eliminare e quali frasi rimuovere, applicato prima della distillazione generica; una regola malformata non blocca nulla. Un classificatore di paywall riconosce gli articoli riservati agli abbonati richiedendo *due* condizioni (testo troppo breve *e* marcatore di paywall), per evitare falsi positivi: un articolo così classificato non viene arricchito, così al posto dell'articolo non si mostra mai il testo promozionale.

**Stadio Readability e scorciatoia JSON-LD.** Poiché ogni nuovo layout richiederebbe una regola nuova, la pipeline adotta anche Readability come stadio generico: se la pagina espone il testo ufficiale in un blocco JSON-LD (`articleBody`), lo si usa direttamente; altrimenti la pagina è caricata in una webview nascosta dove Readability estrae il corpo, con fallback alla pipeline precedente se l'estrazione rende troppo poco.

## 7.6 Feed per argomenti e feed personalizzato

Oltre all'ordine cronologico, la Home offre una vista che *raggruppa per argomento* le notizie delle ultime 24 ore. Il segnale di rilevanza è quello classico degli aggregatori: quante testate distinte coprono la stessa storia. Il raggruppamento avviene sul dispositivo, senza chiamate esterne: i titoli sono ridotti a vettori TF-IDF (dopo normalizzazione e rimozione delle stopwords) e confrontati con la similarità coseno Manning et al., 2008; nel dubbio si preferisce non unire. Quando è configurato un provider AI, il raggruppamento passa prima da lui: il modello riceve i titoli e risponde in JSON con i gruppi, validati difensivamente, e coglie somiglianze semantiche che il confronto lessicale non vede. Qualunque fallimento ricade in silenzio sul clustering lessicale, così il feed non degrada mai per colpa del provider. L'interfaccia rende esplicita l'aggregazione (etichetta di

copertura, articolo guida, sintesi AI dell'intera storia nella lingua scelta) e la persiste per 12 ore, ricalcolandola su trascinamento. Il clustering su embedding resta l'evoluzione prevista: cambierebbe il vettorizzatore, non la forma della pipeline.

### 7.6.1 Onboarding e feed “Per te”

L'onboarding è in quattro passi: introduzione, scelta delle testate (senza scaricare articoli), scelta degli *interessi* (etichette suggerite più campo libero) e configurazione facoltativa del provider AI. Gli interessi alimentano il feed “Per te”, terzo segmento della Home. La scelta di progetto è netta: la personalizzazione è *solo AI*, senza surrogati lessicali. Se manca il provider o mancano gli interessi la schermata lo dichiara e rimanda alle Impostazioni, senza mai spacciare un elenco qualunque per personalizzato. Il modello riceve titoli e interessi e risponde con un insieme ordinato di scelte, ognuna con l'interesse che l'ha promossa e una rilevanza; il risultato è validato (identificatori sconosciuti e doppioni scartati) e persistito per 12 ore, invalidato quando gli interessi cambiano. Gli interessi sono modificabili anche dalle Impostazioni.

## 7.7 Impostazioni, design system e osservabilità

**Configurazione consumer del provider AI.** La configurazione del provider AI, all'inizio confinata agli strumenti sviluppatore, è promossa nelle Impostazioni: una schermata elenca i cinque provider con lo stato (pronto / manca la chiave), attiva il provider al tocco, accetta la chiave in un campo cifrato (mai rimostrata), gestisce l'endpoint locale di Ollama e carica l'elenco dei modelli. La schermata riusa un unico view model condiviso con gli strumenti sviluppatore: un solo comportamento, due superfici.

**Design system “Paper”.** L'interfaccia consumer adotta un design system ispirato alla carta stampata (sfondo carta e inchiostro adattivi al tema scuro, caratteri con grazie per i titoli, monospazio per i metadati, filetti ed etichette maiuscole al

posto di ombre e card). I token vivono in un unico file e la relativa guida è nella catena di lettura obbligatoria: ogni intervento di interfaccia deve usare i token e non lo stile di default di iOS.

**Gating dell'onboarding e cache preferenze-aware.** All'avvio l'app distingue tre stati (risposta non ancora arrivata, onboarding da fare, completato) e non scarica nulla finché l'onboarding non è concluso: il primo download parte solo dopo la conferma e usa esattamente le fonti scelte. La cache è filtrata sulle preferenze (ogni articolo porta l'identificativo della sua fonte), con pulizia immediata degli articoli delle fonti disattivate (mai i preferiti) e retention a 30 giorni; le operazioni pesanti su `SwiftData` girano fuori dal main thread per non introdurre micro-blocchi all'avvio.

**Sintesi AI su comando.** La sintesi di un articolo è sempre esplicita, avviata da un bottone, mai automatica all'apertura. Vale anche quando esiste una sintesi in cache, che il tap mostra istantaneamente senza chiamare il provider. Per evitare che l'estratto RSS venga scambiato per una sintesi AI, l'output del modello vive in campi dedicati (`aiSummaryShort`, `aiSummaryBullets`) scritti solo dal flusso di sintesi.

**Logging strutturato.** Ogni funzione nuova o modificata emette log via `AppLogger` (timestamp, livello, categoria, modulo, file/linea, funzione, request-id, metadati), con propagazione del request-id nei flussi multi-layer; i log sono esportabili dal Developer Playground e non contengono segreti né contenuto utente privato.



# Capitolo 8

## Testing e validazione

### 8.1 Strategia di test

La strategia prevede due livelli. Gli **unit test** (*Swift Testing*) coprono modelli di dominio, use case, servizi, repository, parser, mapper, normalizzazione delle risposte dei provider e ViewModel; gli **UI test** (*XCTest*) sono riservati ai pochi flussi end-to-end ad alto valore (avvio, onboarding, caricamento feed, dettaglio articolo) ed evitati per modifiche di solo layout o copia. La regola operativa è che ogni cambiamento di comportamento aggiorni i test al livello adeguato, oppure ne motivi esplicitamente l'assenza.

### 8.2 Copertura

La copertura si concentra dove il rischio di regressione è maggiore: il parsing RSS e la normalizzazione, la mappatura delle risposte dei provider AI, le macchine a stati dei ViewModel e degli use case, il ranking del feed (sez. 7.4) e la pipeline di distillazione. Quest'ultima è validata da una suite parametrica su un corpus di pagine reali delle testate italiane, con controlli espliciti sul contenuto che deve sopravvivere e sul chrome che deve sparire; aggiungere una testata è un rituale documentato (nuova fixture, nuova riga di attesa, riesecuzione della suite). Le superfici di aggregazione e personalizzazione sono coperte da test deterministici sul clustering lessicale, sulla validazione delle risposte AI (gruppi e scelte perso-

nalizzate) e sulla reidratazione delle cache a 12 ore, inclusi gli stati di assenza di provider o di interessi.

### **8.3 Validazione qualitativa**

La validazione qualitativa si è appoggiata al Developer Playground, che ha permesso di esercitare i parser su decine di feed reali e di isolare classi di problemi (URL canonici incoerenti, date malformate, snippet troncati, corpi parziali) iterando sulle euristiche di normalizzazione ed estrazione. Lo stesso strumento isola il layer AI dal flusso consumer, così da distinguere una regressione del provider da un problema dell'interfaccia.

# Capitolo 9

## Conclusioni

### 9.1 Risultati raggiunti

Mercury raggiunge gli obiettivi MVP della tesi: una pipeline RSS robusta con superfici separate per produzione e diagnostica (PR #38), una persistenza locale-first su SwiftData con store dedicato `@ModelActor` e servizio preferenze `@MainActor` (PR #34, #35), un layer AI plug-in con cinque provider intercambiabili (PR #23, #39) reso robusto da un fix centralizzato dell'invariante di parsing (PR #42), una Home consumer-facing agganciata al feed reale (PR #36) e il primo flusso di lettura completo con `ArticleDetailScreen` e slot di enrichment AI predisposto (PR #40), un Developer Playground esteso con il runner per-feature del layer AI (PR #41), una strumentazione di diagnostica e logging completa, e una governance documentale che ha permesso di sviluppare l'intero sistema con un modello human-as-architect, AI-as-implementer. Il catalogo RSS, costruito con un artefatto di ricerca versionato (sezione 7.1), è stato validato dal vivo e potato a 153 feed sani (sezione 7.1). Il ciclo conclusivo sulla qualità del lettore ha aggiunto le regole di estrazione per-outlet, il classificatore paywall, lo stadio Readability con scorciatoia JSON-LD e la gestione della cache (sezioni 7.5, 7.5 e 7.7); coerentemente, lo scope del prodotto è stato ristretto alle sole testate italiane e l'app rinominata Mercurio (issue #102, sezione 7.1), portando lo strato dati e l'esperienza di lettura al livello richiesto per la demo. L'arco finale ha completato l'esperienza consumer: il design system "Paper" vincolante (sez. 7.7),

la Home a tre viste con l'aggregazione per argomenti (AI col fallback lessicale, cache di 12 ore e sintesi della storia, sez. 7.6), la configurazione consumer del provider AI (sez. 7.7) e l'onboarding in quattro passi con il feed personalizzato “Per te” sugli interessi dichiarati (sez. 7.6.1), il tutto indurito da quattro cicli di debugging dal vivo sul dispositivo con console agganciata.

## 9.2 Limiti

La personalizzazione copre gli interessi dichiarati esplicitamente (feed “Per te”, sez. 7.6.1) ma non ancora i segnali di lettura impliciti (aperture, tempi di lettura, bookmark), che restano a livello di specifica; è inoltre vincolata di proposito a un provider AI configurato, senza surrogato locale. Il clustering è implementato nella variante lessicale on-device con raggruppamento AI opzionale (sez. 7.6), mentre la versione su embedding resta specifica; la sincronizzazione cloud non è inclusa nell'MVP. Lo scope del prodotto è limitato alle testate italiane (issue #102): è una scelta deliberata, che concentra la qualità dove è stata misurata, ma limita la generalità dell'app.

## 9.3 Sviluppi futuri

Le direzioni naturali di prosecuzione sono: (i) estensione del feed “Per te” ai segnali di lettura impliciti già modellati (`InteractionEntity`), mantenendo il ranking ispezionabile; (ii) evoluzione del clustering lessicale (sez. 7.6) verso embedding generati lato provider o tramite `Natural Language`; (iii) sincronizzazione opzionale del profilo via Firebase; (iv) notifiche intelligenti basate sui topic preferiti; (v) riattivazione del catalogo multi-regione (già nel codice dietro il flag `MERCURY_MULTI_REGION`), estendendo alle nuove testate lo stesso ciclo di audit e regole applicato a quelle italiane.

# Bibliografia

- Apple Inc. (2024a). Core ML — Developer Documentation [Consultato: luglio 2026].
- Apple Inc. (2024b). SwiftData — Developer Documentation [Consultato: luglio 2026].
- Apple Inc. (2024c). SwiftUI — Developer Documentation [Consultato: luglio 2026].
- Blondel, V. D., Guillaume, J.-L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10), P10008.
- Campello, R. J. G. B., Moulavi, D., & Sander, J. (2013). Density-Based Clustering Based on Hierarchical Density Estimates. *Advances in Knowledge Discovery and Data Mining (PAKDD)*, 160–172.
- Conventional Commits. (2023). Conventional Commits 1.0.0 [Consultato: luglio 2026].
- DSDM Consortium. (2014). MoSCoW Prioritisation — The DSDM Agile Project Framework [Consultato: luglio 2026].
- Feedly. (2024). Feedly AI [Consultato: luglio 2026].
- Hedley, J. (2024). jsoup: Java HTML Parser [SwiftSoup ne è il port Swift. Consultato: luglio 2026].
- Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12), 1–38.
- Kleppmann, M., Wiggins, A., van Hardenberg, P., & McGranaghan, M. (2019). Local-first software: you own your data, in spite of the cloud. *Proceedings*

*of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*, 154–178.

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.

Mozilla. (2024). Readability.js — the standalone library behind Firefox Reader View [Consultato: luglio 2026].

Nottingham, M., & Sayre, R. (2005). *The Atom Syndication Format* (RFC N. 4287). IETF. <https://doi.org/10.17487/RFC4287>

RSS Advisory Board. (2009). RSS 2.0 Specification [Consultato: luglio 2026].

Simmons, B., & NetNewsWire contributors. (2024). NetNewsWire: a free and open source RSS reader for macOS and iOS [Consultato: luglio 2026].

Sommerville, I. (2016). *Software Engineering* (10<sup>a</sup> ed.). Pearson.